

ZIPBrain: Can EEG Foundation Models Be Faster, Locally Deployable, but Accurate?

Lingwei Li¹, Yirong Kan¹, Peng Chen², Xu Cao³, Zheng Chen⁴, Yasuhiko Nakashima¹

¹Nara Institute of Science and Technology, Nara, Japan

²RIKEN Center for Computational Science, Hyogo, Japan

³University of Illinois Urbana-Champaign, USA

⁴SANKEN, The University of Osaka, Japan

li.lingwei.lo2@naist.ac.jp, kan.yirong@is.naist.jp, peng.chen@riken.jp, xucao2@illinois.edu, chenz@sanken.osaka-u.ac.jp, nakashim@is.naist.jp

Abstract

This work investigates whether Electroencephalograph (EEG) foundation models (EFMs) can be made faster and locally deployable without sacrificing accuracy. EEG foundation models are a major trend, offering strong general-purpose representations. However, their computational burden grows quadratically with input length, hindering deployment on resource-constrained scenarios, particularly for real-time clinical monitoring. EEG’s low SNR further suggests many of these tokens are redundant and compressible with little accuracy cost. We propose ZIPBrain, a novel redundancy-aware EEG token pooling module that leverages this low-SNR characteristic to reduce token count. Given a token sequence, ZIPBrain partitions tokens into redundant and unique groups, then merges each redundant token with its most similar counterpart in the unique group. Furthermore, ZIPBrain serves as a training-free, plug-and-play module that seamlessly integrates into standard Transformer encoders with negligible computational overhead. Extensive experiments across multiple EEG foundation models show ZIPBrain’s strong versatility, achieving 1.3%-10.5% average improvement over baselines, while reducing wall-clock inference time by 32.7% (up to 41.8% with CUDA Graph) compared to the original foundation model.

Introduction

The breakthroughs in Large Language Models (LLMs) (Radford et al. 2019; Brown et al. 2020; OpenAI 2024; Gemini Team 2025) have driven a broader shift toward scaling attention-based architectures, giving rise to a series of EEG foundation models that push the state of the art on downstream tasks such as seizure detection, Brain-Computer Interfaces (BCIs), and sleep staging (Ma et al. 2025; Pradeepkumar et al. 2025; Yang et al. 2026). Transformers (Vaswani et al. 2017; Devlin et al. 2019), which are backbones of LLMs, require discrete token sequences as input. EEG signals are therefore segmented into patches, which are then embedded into tokens; self-attention operates over these tokens to capture dependencies across time and channels.

However, efficiency remains a critical bottleneck: computing pairwise correlations across all tokens gives Transformers quadratic complexity in sequence length, both in computation and deployment. This burden is particularly acute for EEG, where long-duration or high-density recordings can easily yield token sequences of several thousand in length,

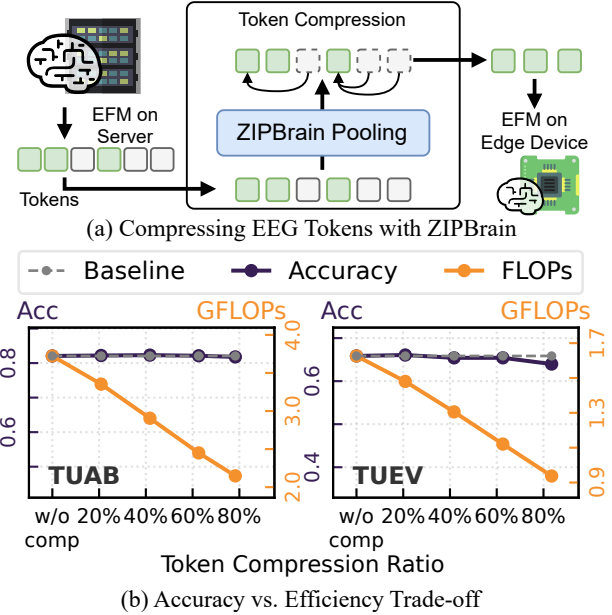


Figure 1: (a) The proposed method ZIPBrain can compress tokens of EFM on server and deploy EFM efficiently on local devices. (b) Our method achieved 42% reduction of computations with negligible impact on accuracy.

restricting deployment in resource-constrained or real-time clinical monitoring scenarios. The problem is compounded by the low signal-to-noise ratio (SNR) intrinsic to EEG data (Chen et al. 2025b): genuine neural activity is often buried beneath physiological artifacts and background noise, so a substantial portion of the resulting tokens carries largely non-discriminative information rather than task-relevant signal.

A few lines of research have been proposed to mitigate efficiency bottleneck and redundancy issue. Some studies propose to reduce the token count right at the data preprocessing stage by selectively sampling patches (Ankireddy et al. 2025; Zhou et al. 2026). Another line bypasses the quadratic complexity entirely by substituting the Transformer architecture with SSMs (Tegon et al. 2025). However, the objective of the above works is data preprocessing rather than token reduc-

tion. In this work, we investigate whether token compression can be applied to EEG foundation models to mitigate efficiency bottleneck by leveraging the low-SNR property of EEG signals. Token compression have been studied in Computer Vision domain, including importance-based compression and redundancy-based compression. Importance-based token compression (Liang et al. 2021; Zhang et al. 2025) focus on identifying informative tokens to preserve, contrastively, emerging in recent years, redundancy-based token compression (Wen et al. 2025; Fang et al. 2025) focus on reducing tokens containing redundancy information. However, we acknowledge that leveraging these methods to EEG foundation models is non-trivial and largely unexplored; and we still face three major challenges: (1) Token Selection. EEG tokens carry no explicit visual or semantic cues. Unlike CV or NLP, there is no clear signal for which tokens matter most. (2) Redundant Token Identification. EEG signals have a low signal-to-noise ratio. Many tokens are therefore redundant, not task-irrelevant. Quantifying this redundancy remains largely unexplored. (3) Merging for Compression. Naively averaging redundant tokens with their target counterparts dampens feature magnitude. An averaged vector’s norm is bounded by its constituents’ maximum. This risks losing salient EEG signatures.

To tackle these challenges, we present ZIPBrain (as shown in Figure 1(a)), a plug-and-play, training-free token pooling framework for EEG foundation models. Specifically, we introduce an l_2 -norm-based pivot selection mechanism to identify a representative token set without relying on CV- or NLP-specific priors; define an explicit redundancy score, computed as each token’s accumulated cosine similarity to the selected pivots, to partition tokens into unique and redundant groups; and merge matched tokens through a norm-preserving pooling operation that rescales the averaged representation to the maximum constituent norm, avoiding the feature attenuation inherent to naive averaging. As shown in Figure 1(b), ZIPBrain preserved 99.65% and 97.14% of LaBraM’s downstream accuracy on TUAB and TUEV dataset at $\sim 80\%$ compression ratio, with 42.30% and 42.36% reduction in computational burdens, and improved the accuracy by 0.2% at $\sim 40\%$ compression ratio on TUAB. In empirical evaluations across four EEG foundation models and five datasets, ZIPBrain achieved 16 top-1 results and 20/20 top-2 results out of 20 settings compared to baselines.

Related Work

EEG Foundation Models. Trained as a foundation model, BIOT (Yang, Westover, and Sun 2023) validated that the pre-training paradigm yields superior representations compared to supervised training methods. Subsequent frameworks, including LaBraM (Jiang, Zhao, and Lu 2024), EEGPT (Wang et al. 2024a), CBraMod (Wang et al. 2024b), TFM-Tokenizer (Pradeepkumar et al. 2025), CodeBrain (Ma et al. 2025), and ST-EEGFormer (Yang et al. 2026), introduced diverse pre-training objectives and tokenization schemes tailored to EEG dynamics, consistently establishing new state-of-the-art benchmarks. However, the deployment of these large-scale models is often constrained by heavy computational overhead, forcing a strategic trade-off

between model performance and computational efficiency.

Token Compression. The token compression has been widely explored in CV to reduce computational complexity by reducing the number of tokens. Existing approaches can be broadly categorized into *importance-based* and *redundancy-based* strategies. *Importance-based methods* identify and reduce the number of uninformative tokens by evaluating token significance using auxiliary predictor networks (Rao et al. 2021), [CLS] attention scores (Liang et al. 2021), or cross-modal attention maps in VLMs (Chen et al. 2025a; Zhang et al. 2025; Ye et al. 2025; Arif et al. 2025). Conversely, *redundancy-based methods* focus on reduce the number of redundant tokens, employing techniques such as bipartite soft matching (Bolya et al. 2023) or explicit redundancy metrics (Fang et al. 2025; Dhouib et al. 2025).

-Issues. Despite its proved success in visual domains, token compression in the context of EEG Transformers remains largely under-explored. EEG signals exhibit fundamentally different characteristics from images, transferring vision-oriented compression strategies to EEG models naively may risk discarding critical neural information, resulting a retrogradation in model capability.

Problem Definition

This section presents the problem formulation of token compression for EEG foundation models.

Definition 1 (Multi-Channel EEG Tokens). EEG recordings capture neuronal activity across multiple brain regions over time. Let $\mathbf{S} \in \mathbb{R}^{C \times T}$ denote an EEG recording, where C is the number of channels and T is the number of samples.

The recording $\mathbf{S}$ is segmented using a sliding window of size w with stride s , producing $\hat{\mathbf{S}} \in \mathbb{R}^{C \times \tau \times w}$, where $\tau = \lfloor \frac{T-w}{s} \rfloor + 1$ is the number of temporal segments. Each segment is then projected into a d -dimensional embedding, yielding the initial token sequence $\mathbf{X}^0 = \{\mathbf{x}_i^0\}_{i=1}^N \in \mathbb{R}^{N \times d}$, where $\mathbf{x}_i^0 \in \mathbb{R}^d$ denotes the i -th token, $N = C\tau$ is the total number of tokens, and d is the embedding dimension. The superscript l denotes the Transformer layer, such that $\mathbf{X}^0$ represents the input tokens and $\mathbf{X}^l$ the token representations at the l -th layer.

Definition 2 (Token Pooling). Given a sequence of tokens, token pooling aims to merge tokens together

$$[\tilde{X}_1, \dots, \tilde{X}_{n_2}] := f_{\text{pool}}(X_1, X_2, \dots, X_{n_1}), \quad (1)$$

where $n_2 < n_1$ and f_{pool} can be any operation that implements this behavior.

However, implementing f_{pool} is non-trivial for the preservation of model’s capability, with following three problems.

Problem 1 (Token Selection). One of the core challenges in token merging is to determine an optimal partition of the input tokens into N' groups, denoted by $\mathcal{G} = \{\mathbf{G}_1, \mathbf{G}_2, \dots, \mathbf{G}_{N'}\}$, where $\mathbf{G}_j$ is the j -th token group. Given the input token sequence $\mathbf{X} \in \mathbb{R}^{N \times d}$ and the target sequence length N' , the grouping can be represented by a binary assignment matrix $\mathbf{M} = \{m_{ij}\} \in \{0, 1\}^{N \times N'}$, where

$$\mathbf{G}_j = \{\mathbf{x}_i \mid m_{ij} = 1\}.$$

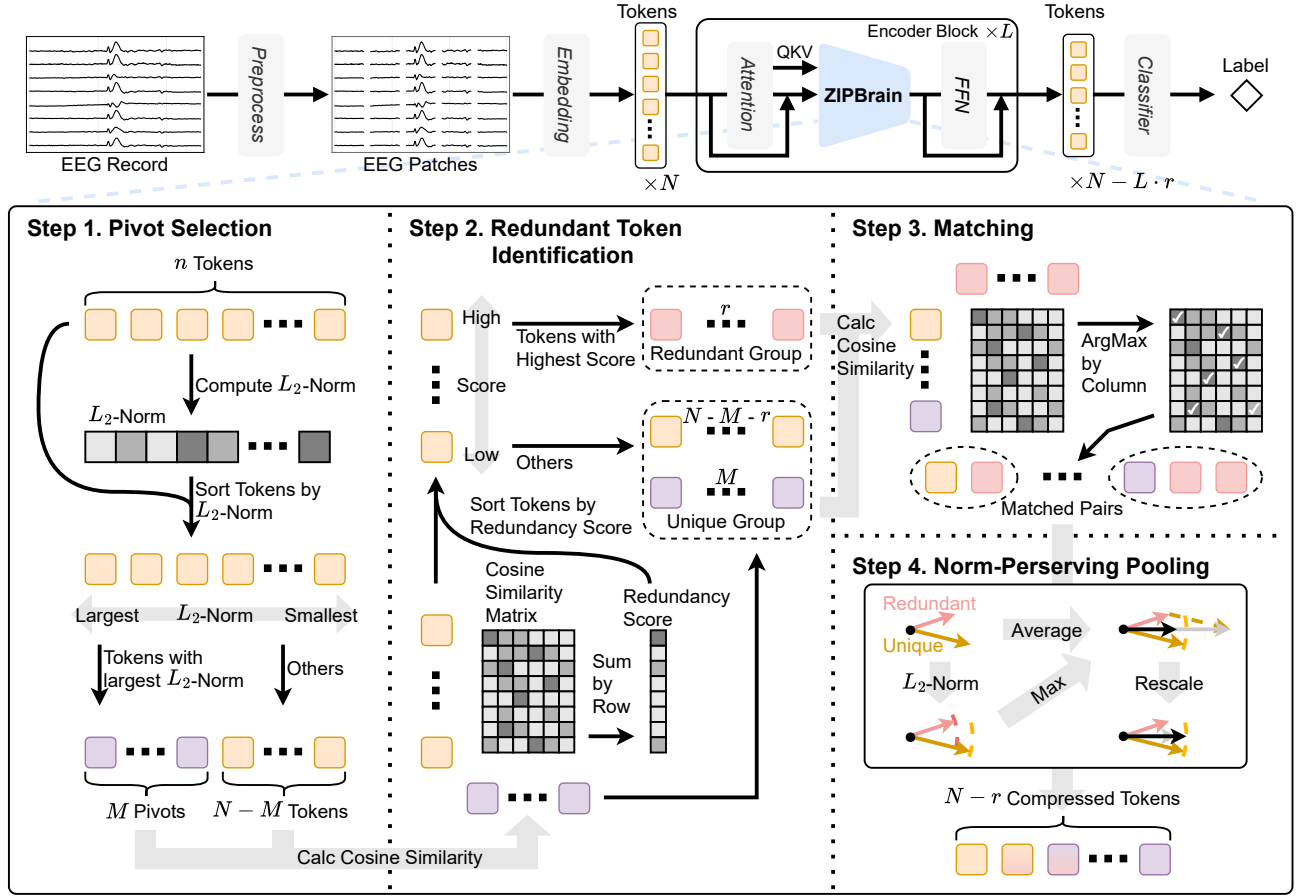


Figure 2: Overview of the proposed module ZIPBrain. The flow-chart in the top illustrates the integration of our method into a Transformer block, positioned between the Attention and Feed-Forward layers to leverage intermediate Q , K , or V representations. The bottom panel details three processes of the method: (1) *Redundancy-based partitioning*, where tokens are partitioned into unique and redundant sets based on redundancy scores calculated relative to pivots, which are selected from the input tokens by ranking their ℓ_2 -norms in descending order and assigned to unique group; (2) *Similarity-based matching*, where each redundant token is matched with its most similar counterpart in unique group; and (3) *Norm-preserving pooling*, which includes a rescaling operation to preserve the original norm characteristics of the tokens, following taking average of tokens.

Here, $m_{ij} = 1$ indicates that $\mathbf{x}_i$ is assigned to group j , subject to the partition $\sum_{j=1}^{N'} m_{ij} = 1$, $\forall i \in \{1, \dots, N\}$, ensuring that each token is assigned to one group.

Problem 2 (Redundancy of EEG Tokens). In the context of EEG processing, tokens naturally exhibit substantial redundancy. Effectively exploiting this redundancy to guide token grouping remains a fundamental challenge in token compression. Formally, given the token sequence $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^N$, the objective is to define a redundancy measure $r(\mathbf{x}_i, \mathbf{X})$, which quantifies the redundancy of given token $\mathbf{x}_i$ in population $\mathbf{X}$ and guides the construction of the token partition $\mathcal{G}$.

Problem 3 (Merging Strategy). Given a token group $\mathbf{G}_j$, the final challenge is to define a merging function f_{merge} that aggregates all tokens within the group into a single representative token,

$$\hat{\mathbf{x}}_j = f_{\text{merge}}(\{\mathbf{x} \mid \mathbf{x} \in \mathbf{G}_j\}). \quad (2)$$

Various choices exist for implementing f_{merge} , ranging from simple mean pooling to more sophisticated attention-based aggregation strategies.

Method

Overview of ZIPBrain

To tackle the above challenges, we propose ZIPBrain, a plug-and-play token compression method for Transformer-based EEG foundation models. As illustrated in the top panel of Figure 2, ZIPBrain is seamlessly inserted between the self-attention module and the feed-forward network (FFN), enabling token compression without modifying the Transformer backbone. Given the intermediate tokens produced by the attention, ZIPBrain compresses them through three sequential components, as shown in the bottom panel of Figure 2: (1) a two-stage partitioning module (Step 1 and 2) that partitions tokens into two sets, with a redundancy-aware partitioning strategy that exploits the intrinsic redundancy

of EEG tokens, (2) a token matching module (Step 3) that further construct groups by identifying merging target for redundant tokens, and (3) a norm-preserving merging module (Step 4) that aggregates each group into a representative token while preserving informative representations. Together, these components effectively reduce the sequence length while maintaining the discriminative information required for downstream EEG analysis.

Step 1: Pivots Selection

The two-stage partitioning module categorizes tokens into a redundant set and a unique set based on their redundancy. This module begins with a Pivot Selection stage, which selects a subset of tokens—termed pivots—to represent the collective information of the original tokens. Specifically, the pivots $\mathbf{X}_p$ are chosen as the tokens with the largest ℓ_2 -norms. We use $\mathcal{I}_p$ to represent the indices set of pivots, define

$$\mathcal{I}_p = \arg \max_{S \subseteq \{1, \dots, N-r\}, |S|=N_p} \sum_{i \in S} \|\mathbf{x}_i\|_2, \quad (3)$$

and define $\mathbf{X}_p$ as $\mathbf{X}_p = \mathbf{x}_{i \in \mathcal{I}_p}$ to represent the set of pivotal tokens. The $N_p = \lceil (N-r) \cdot \rho \rceil$ denotes the number of selected pivots, and is controlled by a hyperparameter ratio $\rho \in (0, 1]$ that ensures $N_p \in [1, N-r]$.

Step 2: Redundant Token Identification

Since pivots capture the core contextual information, a token's redundancy naturally corresponds to its alignment with these pivots. Formally, we evaluate the redundancy score s_i for each token $\mathbf{x}_i$ by aggregating its similarities across all pivots:

$$s_i = \begin{cases} 0, & \text{if } i \in \mathcal{I}_p, \\ \sum_{j \in \mathcal{I}_p} \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_j^\top, & \text{otherwise,} \end{cases} \quad (4)$$

where $\tilde{\mathbf{x}}_i$ and $\tilde{\mathbf{x}}_j$ denote the ℓ_2 -normalized forms of $\mathbf{x}_i$ and $\mathbf{x}_j$, respectively, and $\tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_j^\top$ denotes the cosine similarity between candidate token $\mathbf{x}_i$ and pivot token $\mathbf{x}_j$. Assuming row vectors $\mathbf{x}_i \in \mathbb{R}^{1 \times d}$, the matrix product $\tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_j^\top$ produces a scalar. The summation $\sum_{j \in \mathcal{I}_p} \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_j^\top$ measures the cumulative cosine similarity and can be efficiently computed by associative reordering: $\tilde{\mathbf{x}}_i \left(\sum_{j \in \mathcal{I}_p} \tilde{\mathbf{x}}_j^\top \right)$. Notably, pivots are assigned a zero score to ensure their preservation during token compression.

Tokens are assigned to either the redundant set or the unique set depending on their redundancy scores. Formally, we denote the redundant and unique token sets as $\mathbf{X}_r$ and $\mathbf{X}_u$, respectively, with corresponding index sets $\mathcal{I}_r$ and $\mathcal{I}_u$, defined as $\mathbf{X}_r = \{\mathbf{x}_i \mid i \in \mathcal{I}_r\}$ and $\mathbf{X}_u = \{\mathbf{x}_i \mid i \in \mathcal{I}_u\}$. Tokens with the top- r redundancy scores are assigned to the redundant set $\mathbf{X}_r$, while the remaining tokens are assigned to the unique set $\mathbf{X}_u$. Therefore, the index sets $\mathcal{I}_r$ and $\mathcal{I}_u$ are defined as:

$$\begin{aligned} \mathcal{I}_r &= \arg \max_{S \subseteq \{1, \dots, N\}, |S|=r} \sum_{i \in S} s_i, \\ \mathcal{I}_u &= \{1, \dots, N\} \setminus \mathcal{I}_r, \end{aligned} \quad (5)$$

where $\setminus$ denotes the set difference operation.

Step 3: Matching For Token Pairs.

For each redundant token $\mathbf{x}_i \in \mathcal{I}_r$, we match it with the most similar counterpart in unique set and use function $\mathcal{M}(i \in \mathcal{I}_r) \rightarrow j \in \mathcal{I}_u$ to formulate this projection as

$$\mathcal{M}(i) = \arg \max_{j \in \mathcal{I}_u} (\tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_j^\top), \forall i \in \mathcal{I}_r, \quad (6)$$

where $\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j$ denote ℓ_2 -normalized $\mathbf{x}_i, \mathbf{x}_j$ and $\tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_j^\top$ calculated the cosine similarity between two vectors. Groups are then defined on top of $\mathcal{M}(\cdot)$ as

$$\begin{aligned} \mathcal{G} &= \{\mathbf{G}_j, j \in [1, \dots, N']\} \\ &= \{\{\mathbf{x}_i\} \cup \{\mathbf{x}_j \mid \mathcal{M}(j) = i, j \in \mathcal{I}_r\}, \forall i \in \mathcal{I}_u\}, \end{aligned} \quad (7)$$

which provides the necessary pairing information for the next token merging step.

Step 4: Norm-Preserving Feature Merging

The merging mechanism operates on the principle of norm preservation across all tokens within $\mathbf{G}_j$. It first calculates the average direction of the cluster tokens and then scales this normalized vector by the maximum feature norm observed among the individual vectors in the set:

$$\bar{\mathbf{x}}_j = \frac{1}{|\mathbf{G}_j|} \sum_{\mathbf{x}_k \in \mathbf{G}_j} \mathbf{x}_k, \quad (8)$$

$$\tilde{\mathbf{x}}_j = \frac{\bar{\mathbf{x}}_j}{\|\bar{\mathbf{x}}_j\|_2} \cdot \max_{\mathbf{x}_k \in \mathbf{G}_j} \|\mathbf{x}_k\|_2, \quad (9)$$

where $\tilde{\mathbf{x}}_j \in \mathbb{R}^d$ represents the final condensed token representation that forms the j -th row of the compressed feature matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{N' \times d}$.

By adopting this norm-preserving pooling strategy, our framework avoids the erosion of feature magnitude that inherently occurs with standard linear averaging, where the norm of the average vector is strictly bounded by the maximum norm of its constituents (i.e., $\|\bar{\mathbf{x}}_j\|_2 \leq \max_{\mathbf{x}_k \in \mathcal{X}_j} \|\mathbf{x}_k\|_2$). This design ensures that the fused tokens maintain a feature energy consistent with the original latent distribution, thereby preserving the representational strength of critical EEG signatures throughout the compression sequence.

Representation Space Selection

The proposed ZIPBrain is representation-agnostic and can operate on different latent representations within the Transformer block. Specifically, the three stages of ZIPBrain—pivot selection, redundancy estimation, and token grouping—can be performed using either the post-attention representation $\hat{\mathbf{X}}$ or the Query, Key, and Value projections $\{\mathbf{Q}, \mathbf{K}, \mathbf{V}\}$. Different representation spaces emphasize different characteristics of the token interactions and therefore lead to different grouping behaviors. Rather than fixing a particular representation, ZIPBrain treats the representation space as a modular design choice. The optimal representation is selected through hyperparameter optimization according to the backbone architecture and downstream task.

Model	Methods	TUAB		EEGMAT		TUEV		ISRUC		EarEEG	
		AUROC	PR-AUC	AUROC	PR-AUC	Kappa ²	W-F1 ²	Kappa	W-F1	Kappa	W-F1
LaBraM	Original ¹	0.9067	0.9092	0.6230	0.6244	0.6616	0.8292	0.7383	0.7958	0.4255	0.5413
	EVIT	0.8529	0.8501	0.6342	0.6316	0.5766	0.7922	0.5972	0.6849	0.2474	0.4003
	ToMe	0.8854	0.8893	0.6086	0.6042	0.6227	0.8106	0.6084	0.6951	0.3072	0.4476
	ToFU	0.8884	0.8928	0.6154	0.6234	0.6272	0.8129	0.6332	0.7139	0.3069	0.4497
	DART	0.8731	0.8713	0.6207	0.6077	0.5416	0.7704	0.6642	0.7348	0.3489	0.4805
	Ours	0.8906	<u>0.8916</u>	0.6477	0.6751	0.6479	0.8226	0.6763	0.7508	0.4249	0.5318
EEGPT	Original ¹	0.8684	0.8470	0.6517	0.6211	0.5677	0.7887	0.6587	0.7252	0.4797	0.5767
	EVIT ^{mean}	0.8665	0.8449	0.6512	0.6211	0.5603	0.7861	0.6544	0.7203	0.4558	0.5565
	ToMe	0.8672	0.8457	0.6444	0.6108	0.5557	0.7850	0.6411	0.7111	0.4539	0.5543
	ToFU	0.8672	0.8458	0.6459	0.6120	0.5566	0.7854	0.6441	0.7134	0.4539	0.5550
	DART	0.8687	0.8468	0.6528	0.6201	0.5620	0.7872	0.6637	0.7301	0.4647	0.5634
	Ours	0.8690	0.8482	0.6563	0.6257	0.5746	0.7889	<u>0.6612</u>	<u>0.7273</u>	0.4689	0.5656
BIOT	Original ¹	0.8782	0.8797	0.7721	0.7805	0.5082	0.7444	0.7536	0.8050	0.3935	0.5267
	EVIT ^{mean}	0.8647	0.8673	0.8251	0.8266	0.4649	0.7151	0.5702	0.6643	0.4965	0.5943
	ToMe	0.8767	0.8765	0.7454	0.7425	0.5000	0.7412	0.7334	0.7905	0.4172	0.5325
	ToFU	0.8771	0.8771	0.7410	0.7419	0.5020	0.7423	0.7343	0.7892	0.4101	0.5289
	DART	0.8639	0.8660	0.7255	0.7309	0.4587	0.7138	0.6238	0.7180	0.2705	0.4042
	Ours	0.8797	0.8791	0.8286	<u>0.8149</u>	0.5199	0.7488	0.7345	0.7933	<u>0.4663</u>	<u>0.5642</u>
TFM	Original ¹	0.8895	0.8964	0.6299	0.6210	0.5611	0.7713	0.7094	0.7684	0.4077	0.5250
	EVIT	0.8708	0.8779	0.6075	0.5713	0.4411	0.7155	0.6727	0.7429	0.4436	0.5488
	ToMe	0.8793	0.8834	0.6363	0.6279	0.5421	0.7615	0.6879	0.7487	0.3927	0.5071
	ToFU	0.8793	0.8846	0.6435	0.6526	0.5399	0.7603	0.6921	0.7530	0.3862	0.5016
	DART	0.8624	0.8662	0.5990	0.5889	0.5240	0.7523	0.6870	0.7537	0.2687	0.4227
	Ours	0.8812	0.8888	0.6727	0.6662	0.5628	0.7712	0.7120	0.7641	0.4839	0.5771

Experiment results with r , the number of tokens to be reduced per reduction, set to the biggest integer smaller than $\frac{N}{l}$. N is the number of total tokens and l is set to 4 for BIOT, TFM and EEGPT and 12 for LaBraM. For BIOT and TFM, which consist of 4 layers of encoder, and LaBraM, which consists of 12 layers of encoder, we insert reductions in every layer. For EEGPT, which consists of 8 layer of encoder, reductions are insert at the 2nd, 4th, 6th and 8th layer. All hyper-parameters are set based on recommendations from original works. Compression ratio is defined as $(N - l \cdot r)/N$.

¹ We include experiment results without token compression as Original.

² Kappa here stands for Cohen’s Kappa and W-F1 stands for Weighted F1.

Table 1: Comparison on accuracy with highest compression ratio available

Experiments and Results

Experimental Setup

To evaluate versatility, we integrate our inference-stage, plug-and-play module into four representative EEG foundation models spanning diverse architectures and pretraining objectives: LaBraM (Jiang, Zhao, and Lu 2024), EEGPT (Wang et al. 2024a), BIOT (Yang, Westover, and Sun 2023), and TFM-Tokenizer (Pradeepkumar et al. 2025). Since official checkpoints are largely unavailable, we independently fine-tune all models on downstream tasks. Experiments are conducted on GTX 5090 and GTX 4090 with Ubuntu 2204.

We compare against four CV token compression methods: ToMe (Bolya et al. 2023) (bipartite-matching token merging; official code), ToFU (Kim et al. 2024) (norm-preserving MLERP merging; re-implemented on ToMe), EViT (Liang et al. 2021) ([CLS]-attention-guided pruning; implemented per paper), and DART (Wen et al. 2025) (pivot-based duplicity pruning; modified for controllable compression ratios).

We conduct experiments across five diverse EEG datasets:

Dataset	# of Class	Total Samples	Data Proportion		Bal? ¹	Main Metric
			Max	Min		
TUAB	Binary	36945	53.88%	46.12%	✓	AUROC
EEGMAT	Binary	290	50.00%	50.00%	✓	AUROC
TUEV	6	29421	66.78%	1.12%	×	Kappa ²
ISRUC	5	8700	35.86%	11.85%	×	Kappa ²
EarEEG	6	2944	40.05%	6.93%	×	Kappa ²

Table 2: A summary of datasets and main metric used for each dataset. Bal?¹ shows whether the distribution of labels are balanced or imbalanced; Kappa² stands for Cohen’s Kappa

TUAB (Obeid and Picone 2016): binary normal-vs-abnormal classification (standard 10-20 system); EEGMAT (Zyma et al. 2019): binary resting-vs-arithmetic state classification; TUEV (Obeid and Picone 2016): 6-class seizure type detection; ISRUC-Sleep (Khalighi et al. 2016): 5-stage PSG sleep scoring; and EarEEG (Bjarke Mikkelsen et al. 2025): 6-class

Dataset	Method	20%		40%		60%		80%	
		AUROC	PR-AUC	AUROC	PR-AUC	AUROC	PR-AUC	AUROC	PR-AUC
TUAB (BIOT)	Original ¹	0.8782	0.8797	0.8782	0.8797	0.8782	0.8797	0.8782	0.8797
	ToMe	0.8786	0.8789	0.8786	0.8776	0.8779	0.8760	0.8773	0.8763
	ToFU	0.8787	0.8790	0.8787	0.8777	0.8780	0.8762	0.8775	0.8767
	EViT	0.8773	0.8799	0.8754	0.8789	0.8726	0.8766	0.8691	0.8729
	DART	0.8781	0.8802	0.8773	0.8799	0.8747	0.8777	0.8680	0.8712
	Ours	0.8792	0.8805	0.8801	0.8811	0.8810	0.8812	0.8812	0.8805
Dataset	Method	20%		40%		60%		80%	
		Kappa ²	W-F1 ²	Kappa	W-F1	Kappa	W-F1	Kappa	W-F1
TUEV (LaBraM)	Original ¹	0.6616	0.8292	0.6616	0.8292	0.6616	0.8292	0.6616	0.8292
	ToMe	0.6526	0.8254	0.6493	0.8230	0.6302	0.8141	0.6290	0.8133
	ToFU	0.6526	0.8254	0.6548	0.8251	0.6370	0.8165	0.6397	0.8195
	EViT	0.6664	0.8321	0.6633	0.8308	0.6564	0.8275	0.6240	0.8123
	DART	0.6525	0.8248	0.6338	0.8152	0.6037	0.8003	0.5806	0.7886
	Ours	0.6774	0.8379	0.6735	0.8352	0.6625	0.8284	0.6539	0.8258

¹ We show results without token compression for easier comparison.

² Kappa stands for Cohen’s Kappa and W-F1 stands for Weighted-F1

Table 3: Comparison of accuracy among five token compression methods. This table shows the performance of methods across different compression configurations.

classification on sparse ear-centered electrodes. Following Table 3, for binary tasks (TUAB, EEGMAT), we report Accuracy, Balanced Accuracy, PR-AUC, and AUROC (primary metric). For multi-class tasks with severe class imbalance (TUEV, ISRUC, EarEEG), we report Accuracy, Balanced Accuracy, Weighted-F1, and Cohen’s Kappa (primary metric, robust to chance agreement and skew).

Main Results on Accuracy

Performance under Maximum Compression. Table 1 reports the performance of four EEG foundation models on five datasets under the maximum available compression ratio for each architecture. ZIPBrain consistently outperforms all existing token reduction methods (EViT, ToMe, ToFu, and DART) across almost all settings. While competing approaches exhibit substantial performance degradation under extreme compression, ZIPBrain preserves task-relevant tokens and achieves the best accuracy–efficiency trade-off.

Robustness Across Varying Compression Ratios. Table 3 presents the performance of BIOT on TUAB and LaBraM on TUEV under compression ratios ranging from 20% to 80%. Across all compression ratios, ZIPBrain consistently achieves the best performance. Even at an 80% compression ratio, where competing methods degrade substantially, ZIPBrain maintains stable accuracy, demonstrating strong robustness under aggressive token reduction.

Potential Denoising Effects of Token Pooling. Table 3 further reveals that ZIPBrain frequently surpasses the uncompressed baseline (*Original*). On TUAB with BIOT, it achieves an AUROC of 0.8812 at an 80% compression ratio, exceeding the original model (0.8782). On TUEV with LaBraM, it improves Kappa by up to 2.21% at a 20% com-

Variants	LaBraM / TUAB		BIOT / TUEV	
	AUROC	Δ	Kappa	Δ
Original	0.8903	—	0.5103	—
w/ RND Group	0.8883	(-0.23%)	0.5073	(-0.60%)
w/ RND Match	0.8790	(-1.27%)	0.4927	(-3.47%)
w/ Simple Merge	0.8895	(-0.09%)	0.5043	(-1.19%)

Table 4: Ablation study of different components on TUAB and TUEV datasets. Performance drops compared to the original model are shown in parentheses.

pression ratio. These observations suggest that token pooling may improve representation quality by suppressing redundant or task-irrelevant EEG signals.

Ablation Analysis

Effectiveness of Components. Table 4 presents the ablation study of the proposed components using LaBraM on TUAB and BIOT on TUEV. We individually replace the proposed grouping, matching, and norm-preserving merging modules with random grouping (*w/ RND Group*), random matching (*w/ RND Match*), and simple averaging (*w/ Simple Merge*), respectively. The complete ZIPBrain consistently achieves the best performance across both datasets. Here, replacing the proposed matching strategy results in the largest performance drop (−1.27% AUROC on TUAB and −3.47% Kappa on TUEV), indicating that accurate token matching is the most critical factor for effective token pooling.

Is Pooling Better Than Pruning? Figure 3a compares token pooling with token pruning by directly discarding redundant tokens after two-staged partitioning while keeping all

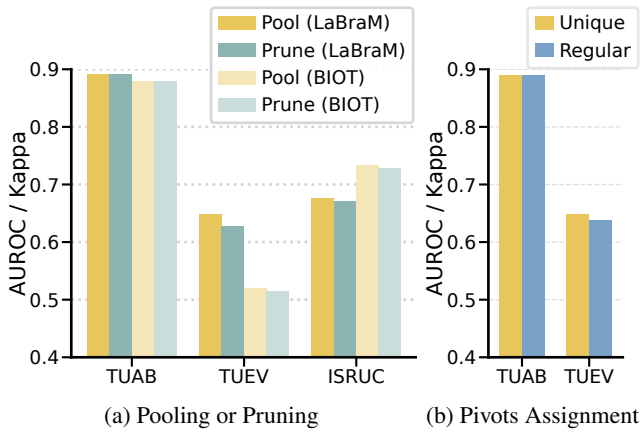


Figure 3: Ablation studies. **(a)** Token pooling consistently outperforms token pruning across all evaluated settings. **(b)** Assigning pivots to the unique group consistently achieves better performance than treating them as regular tokens. AUROC is reported for TUAB, while Cohen’s Kappa is reported for TUEV and ISRUC.

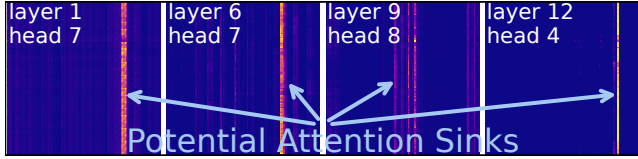


Figure 4: Attention heatmap of LaBraM for TUEV. The vertical lines shown in Attention heatmap implies the existence of Attention Sink.

other components unchanged. On TUAB, the performance gap is marginal, remaining within 0.10% for both LaBraM and BIOT. However, on more challenging datasets (TUEV and ISRUC), pooling consistently outperforms pruning by 0.80%–0.88%, with the largest improvement of 3.04% Cohen’s Kappa on TUEV using LaBraM. These results demonstrate that preserving redundant information through token pooling is more effective than directly pruning tokens.

Is It Safe to Treat Pivots as Unique Tokens? Figure 3b compares our default design, where pivots are assigned to the unique group, with a variant that treats pivots as regular tokens during grouping. Preserving pivots yields better performance, improving AUROC by 0.09% on TUAB and Cohen’s Kappa by 1.47% on TUEV. A possible explanation is that high-norm pivots behave similarly to *attention sinks*, serving as global structural anchors within the Transformer. Merging these tokens with others may dilute their representations and disrupt the attention pattern (Figure 4), whereas preserving them helps maintain model stability.

Deployment Case Study

While accuracy is the gold standard, the practical utility of token compression methods ultimately depends on deployment efficiency. In this case study, we evaluate the runtime performance of ZIPBrain and its compatibility with mod-

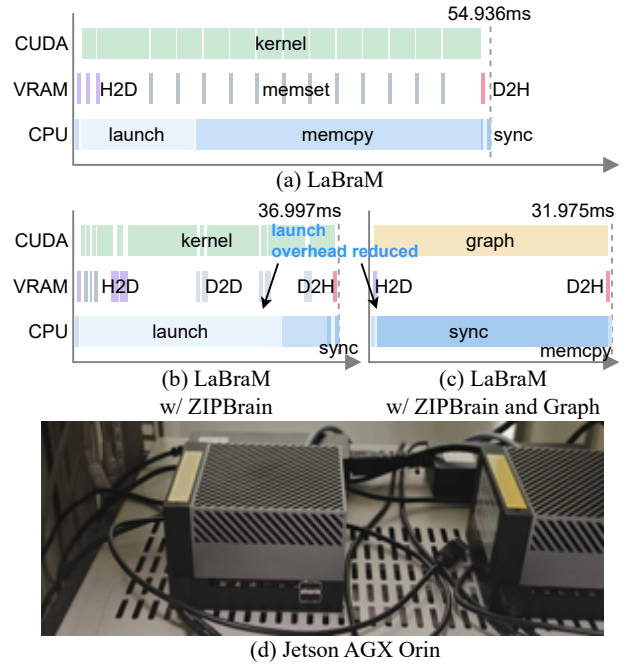


Figure 5: Profiling timeline of LaBraM on Jetson AGX Orin via ONNX Runtime (CUDA). **(a)** Baseline LaBraM takes 54.936 ms. **(b)** Incorporating our proposed ZIPBrain reduces latency to 36.997 ms. **(c)** Combining ZIPBrain with CUDA Graph further drops the total execution time to 31.975 ms. **(d)** The Jetson AGX Orin used in the deployment experiment

ern inference acceleration techniques. Figure 5 compares the runtime profiles under standard eager execution (b) and CUDA Graph optimization (c). Even without graph optimization, ZIPBrain already exhibits high execution efficiency. Owing to its static-shape computation graph, it can be seamlessly accelerated using *CUDA Graphs*, eliminating nearly all kernel launch and synchronization overheads. The optimized implementation achieves a consolidated execution time of 17.18 ms, demonstrating the suitability of our method for efficient edge deployment.

Conclusion

In this work, we presented ZIPBrain, a training-free, plug-and-play token pooling framework that enables efficient inference for EEG foundation models by exploiting the intrinsic redundancy of low-SNR EEG signals. Extensive experiments on four representative EEG foundation models across five benchmark datasets demonstrate that ZIPBrain consistently preserves, and often improves, downstream performance while substantially reducing computational cost. These results suggest that token compression is a practical solution for deploying large-scale EEG foundation models in resource-constrained and real-time clinical settings. Future work will explore combining ZIPBrain with complementary compression techniques, such as weight pruning and quantization, and extending token compression to generative EEG foundation models.

References

- Ankireddy, S. K.; Seleznev, N.; Nguyen, N. H.; Wu, Y.; Kumar, S.; Huang, F.; and Bruss, C. B. 2025. TimeSqueeze: Dynamic Patching for Efficient Time Series Forecasting. In *Recent Advances in Time Series Foundation Models Have We Reached the 'BERT Moment'?*
- Arif, K. H. I.; Yoon, J.; Nikolopoulos, D. S.; Vandierendonck, H.; John, D.; and Ji, B. 2025. HiRED: Attention-Guided Token Dropping for Efficient Inference of High-Resolution Vision-Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 1773–1781.
- Bjarke Mikkelsen, K.; Rezai Tabar, Y.; Rævsbæk Birch, L.; Lind Kappel, S.; Bech Christensen, C.; Dalskov Mosgaard, L.; Otto, M.; Christian Hemmsen, M.; Lind Rank, M.; and Kidmose, P. 2025. Ear-EEG Sleep Monitoring Data Sets. *Scientific Data*, 12(1): 301.
- Bolya, D.; Fu, C.-Y.; Dai, X.; Zhang, P.; Feichtenhofer, C.; and Hoffman, J. 2023. Token Merging: Your ViT But Faster. In *International Conference on Learning Representations*. arXiv.
- Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models Are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, volume 33, 1877–1901. Curran Associates, Inc.
- Chen, L.; Zhao, H.; Liu, T.; Bai, S.; Lin, J.; Zhou, C.; and Chang, B. 2025a. An Image Is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. In Leonardis, A.; Ricci, E.; Roth, S.; Russakovsky, O.; Sattler, T.; and Varol, G., eds., *Computer Vision – ECCV 2024*, volume 15139, 19–35. Cham: Springer Nature Switzerland. ISBN 978-3-031-73003-0 978-3-031-73004-7.
- Chen, Z.; Matsubara, Y.; Sakurai, Y.; and Sun, J. 2025b. Long-Term EEG Partitioning for Seizure Onset Detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 14221–14229.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*, volume 2024, 35549–35562.
- Dao, T.; Fu, D.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, volume 35, 16344–16359.
- Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Burstein, J.; Doran, C.; and Solorio, T., eds., *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. Minneapolis, Minnesota: Association for Computational Linguistics.
- Dhouib, M.; Buscaldi, D.; Vanier, S.; and Shabou, A. 2025. PACT: Pruning and Clustering-Based Token Reduction for Faster Visual Language Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 14582–14592.
- Dremov, A.; Grangier, D.; Katharopoulos, A.; and Hannun, A. 2025. Compute-Optimal Quantization-Aware Training. arXiv:2509.22935.
- Fang, Z.; Lyu, P.; Zhang, C.; Lu, G.; Yu, J.; and Pei, W. 2025. Prune Redundancy, Preserve Essence: Vision Token Compression in VLMs via Synergistic Importance-Diversity. In *The Fourteenth International Conference on Learning Representations*.
- Gemini Team. 2025. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805.
- Jiang, W.-B.; Zhao, L.; and Lu, B.-l. 2024. Large Brain Model for Learning Generic Representations with Tremendous EEG Data in BCI. In Kim, B.; Yue, Y.; Chaudhuri, S.; Fragkiadaki, K.; Khan, M.; and Sun, Y., eds., *International Conference on Learning Representations*, volume 2024, 16405–16426.
- Jiao, X.; Yin, Y.; Shang, L.; Jiang, X.; Chen, X.; Li, L.; Wang, F.; and Liu, Q. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. In Cohn, T.; He, Y.; and Liu, Y., eds., *Findings of the Association for Computational Linguistics: EMNLP 2020*, 4163–4174. Online: Association for Computational Linguistics.
- Khalighi, S.; Sousa, T.; Santos, J. M.; and Nunes, U. 2016. ISRUC-Sleep: A Comprehensive Public Dataset for Sleep Researchers. *Computer Methods and Programs in Biomedicine*, 124: 180–192.
- Kim, M.; Gao, S.; Hsu, Y.-C.; Shen, Y.; and Jin, H. 2024. Token Fusion: Bridging the Gap between Token Pruning and Token Merging. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 1372–1381. Waikoloa, HI, USA: IEEE. ISBN 979-8-3503-1892-0.
- Liang, Y.; Ge, C.; Tong, Z.; Song, Y.; Wang, J.; and Xie, P. 2021. EViT: Expediting Vision Transformers via Token Reorganizations. In *International Conference on Learning Representations*.
- Ma, J.; Wu, F.; Lin, Q.; Xing, Y.; Liu, C.; Jia, Z.; and Feng, M. 2025. CodeBrain: Bridging Decoupled Tokenizer and Multi-Scale Architecture for EEG Foundation Model. In *The Fourteenth International Conference on Learning Representations*.
- Obeid, I.; and Picone, J. 2016. The Temple University Hospital EEG Data Corpus. *Frontiers in Neuroscience*, Volume 10 - 2016.
- OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774.
- Pradeepkumar, J.; Piao, X.; Chen, Z.; and Sun, J. 2025. Tokenizing Single-Channel EEG with Time-Frequency Motif Learning. In *The Fourteenth International Conference on Learning Representations*.
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I.; et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9.

Rao, Y.; Zhao, W.; Liu, B.; Lu, J.; Zhou, J.; and Hsieh, C.-J. 2021. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. In *Advances in Neural Information Processing Systems*, volume 34, 13937–13949. Curran Associates, Inc.

Sun, M.; Liu, Z.; Bair, A.; and Kolter, Z. 2024. A Simple and Effective Pruning Approach for Large Language Models. In Kim, B.; Yue, Y.; Chaudhuri, S.; Fragkiadaki, K.; Khan, M.; and Sun, Y., eds., *International Conference on Learning Representations*, volume 2024, 4942–4964.

Tegon, A.; Ingolfsson, T. M.; Wang, X.; Benini, L.; and Li, Y. 2025. FEMBA: Efficient and Scalable EEG Analysis with a Bidirectional Mamba Foundation Model. In *2025 47th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 1–7.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; ukasz Kaiser, Ł.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.

Wang, G.; Liu, W.; He, Y.; Xu, C.; Ma, L.; and Li, H. 2024a. EEGPT: Pretrained Transformer for Universal and Reliable Representation of EEG Signals. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Wang, J.; Zhao, S.; Luo, Z.; Zhou, Y.; Jiang, H.; Li, S.; Li, T.; and Pan, G. 2024b. CBraMod: A Criss-Cross Brain Foundation Model for EEG Decoding. In *The Thirteenth International Conference on Learning Representations*.

Wang, S.; Li, B. Z.; Khabsa, M.; Fang, H.; and Ma, H. 2020. Linformer: Self-Attention with Linear Complexity. arXiv:2006.04768.

Wen, Z.; Gao, Y.; Wang, S.; Zhang, J.; Zhang, Q.; Li, W.; He, C.; and Zhang, L. 2025. Stop Looking for “Important Tokens” in Multimodal Language Models: Duplication Matters More. In Christodoulopoulos, C.; Chakraborty, T.; Rose, C.; and Peng, V., eds., *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 9961–9980. Suzhou, China: Association for Computational Linguistics. ISBN 979-8-89176-332-6.

Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2024. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438.

Yang, C.; Westover, M.; and Sun, J. 2023. BIOT: Biosignal Transformer for Cross-data Learning in the Wild. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 78240–78260. Curran Associates, Inc.

Yang, L.; Sun, Q.; Li, A.; and Hulle, M. M. V. 2026. Are EEG Foundation Models Worth It? Comparative Evaluation with Traditional Decoders in Diverse BCI Tasks. In *The Fourteenth International Conference on Learning Representations*.

Ye, W.; Wu, Q.; Lin, W.; and Zhou, Y. 2025. Fit and Prune: Fast and Training-free Visual Token Pruning for Multi-modal Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 22128–22136.

Zhang, Q.; Cheng, A.; Lu, M.; Zhang, R.; Zhuo, Z.; Cao, J.; Guo, S.; She, Q.; and Zhang, S. 2025. Beyond Text-Visual Attention: Exploiting Visual Cues for Effective Token Pruning in VLMs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 20857–20867.

Zhou, J.; West, J.; Ehinger, K. A.; Ren, Z.; John, S. E.; and Grayden, D. B. 2026. Adaptive Segmentation of EEG for Machine Learning Applications. *IEEE Journal of Biomedical and Health Informatics*, 1–14.

Zyma, I.; Tukaev, S.; Seleznev, I.; Kiyono, K.; Popov, A.; Chernykh, M.; and Shpenkov, O. 2019. Electroencephalograms during Mental Arithmetic Task Performance. *Data*, 4(14).

Appendix

A Additional Related Work

Efficient Transformers. The computational overhead of large-scale Transformer models has catalyzed several distinct lines of efficiency research. **Quantization** (Dremov et al. 2025; Xiao et al. 2024) compresses models by lowering the numerical precision of weights and activations. **Knowledge distillation** (Jiao et al. 2020) improves inference efficiency by training compact student architectures to mimic the representations of larger teacher models. **Weight pruning** (Sun et al. 2024) eliminates near-zero parameters, which—when paired with sparse storage formats and hardware zero-skipping—alleviates both compute and memory constraints. **Linear attention approximations** (Wang et al. 2020) aim to bypass the quadratic complexity bottleneck, albeit often at the cost of reduced feature dimensionality and diminished model expressiveness. Furthermore, **FlashAttention** (Dao et al. 2022; Dao 2024) represents a hardware-aware breakthrough that accelerates exact attention execution, serving as a fundamental component in modern Transformer architectures.

B Additional Experimental Details

B.1 Dataset Accessibility

All five evaluation benchmarks are publicly accessible online:

- TUAB and TUEV (Obeid and Picone 2016): https://isip.piconepress.com/projects/nedc/html/tuh_eeg/
- ISRUC (Khalighi et al. 2016): <https://sleeptight.isr.uc.pt/>
- EarEEG (Bjarke Mikkelsen et al. 2025): <https://openneuro.org/datasets/ds005178>
- EEGMAT (Zyma et al. 2019): <https://physionet.org/content/eegmat/1.0.0/>

B.2 Data Preprocessing Procedure

The TUAB and TUEV datasets are preprocessed strictly following the scripts provided by their respective baseline implementations, whereas EarEEG adheres to the preprocessing protocol defined in TFM-Tokenizer.

For ISRUC, each epoch is formatted as an individual sample comprising six EEG channels (F3–A2, C3–A2,

O1-A2, F4-A1, C4-A1, O2-A1), using `{_1.txt}` as label files. Data partitions are constructed at the subject level: Subjects 1–80 are allocated to training, 81–90 to validation, and 91–100 to testing.

For EEGMAT, signals are resampled from 500 Hz to 200 Hz and segmented into non-overlapping 2-second patches (with the initial 2 seconds discarded). The subject-level split assigns Subject00–Subject25 to training, Subject26–Subject30 to validation, and Subject31–Subject35 to testing.

B.3 Evaluation Metrics

The evaluation procedures and toolchains are established as follows:

- Accuracy-related classification metrics are computed using `scikit-learn`.
- Computational FLOPs are defined as twice the count of Multiply-Accumulate operations (MACs) and profiled via `onnx-tool` after converting model checkpoints to `onnx` format.
- Inference latency is measured using execution logs captured by NVIDIA Nsight Systems, evaluated with a batch size of 32 over 32 warm-up and measurement iterations.

B.4 Hyperparameters for Fine-Tuning

We fine-tune all four foundation models independently across all five downstream tasks and subsequently evaluate token compression techniques on top of these fine-tuned checkpoints. To ensure maximum reproducibility, we utilize the official code repositories and public checkpoints. The exact hyperparameter configurations used during fine-tuning are summarized in Table 5.

The source code and pre-trained checkpoints are available at the following GitHub repositories:

- BIOT: <https://github.com/ycq091044/BIOT>
- LaBraM: <https://github.com/935963004/LaBraM>
- EEGPT: <https://github.com/BINE022/EEGPT>
- TFM-Tokenizer: <https://github.com/Jathurshan0330/TFM-Tokenizer>

B.5 Baseline Implementations

Official implementations of the three baseline token compression methods are obtained from their public GitHub repositories. Our unified evaluation framework is built upon ToMe’s implementation:

- ToMe: <https://github.com/facebookresearch/ToMe>
- EViT: <https://github.com/youweiliang/evit>
- DART: <https://github.com/ZichenWen1/DART>

B.6 Implementation of the Proposed Method

The complete execution pipeline of ZIPBrain is formally detailed as pseudocode in Algorithm 1.

B.7 Hyperparameter Optimization Procedure

We define the hyperparameter search space for our proposed method as follows:

- **Module Representations (Z_1, Z_2, Z_3):** The tensor representations used in Pivot Selection (Z_1), Redundant Token Identification (Z_2), and Target Token Matching (Z_3) are selected from the candidate set $\{Q, K, V, \hat{X}\}$.
- **Multi-Head Aggregation Strategy:** In multi-head self-attention, aggregating token representations across attention heads is non-trivial. For multi-head tensors (Q, K, V), we evaluate two aggregation strategies: *Pooled Representation* (averaging representations across heads prior to score computation, designated by suffix S) and *Pooled Score* (averaging computed scores across heads, designated by suffix H). Together with the single-tensor $\hat{X}$, each module admits 7 candidate configurations.
- **Pivot Proportion (ρ):** The proportion of retained pivot tokens is governed by $\rho \in (0, 1]$. To make grid search tractable, we discretize this space into 11 candidate values: $\rho \in \{0.0, 0.1, \dots, 1.0\}$. Here, $\rho = 0.0$ represents a boundary case where exactly one pivot token is selected.

The exhaustive search space encompasses $7^3 \times 11 = 3,773$ candidate configurations. To mitigate computational overhead, we adopt a structured two-stage grid search procedure:

1. **Stage 1 (Joint Search for Z_1, Z_2 , and ρ):** We fix the matching representation $Z_3 = \hat{X}$ and perform a grid search over 27 candidate configurations for $\{Z_1, Z_2\}$ across all 11 pivot ratios, forming an initial search space of $27 \times 11 = 297$ combinations. The 27 candidate configurations for $\{Z_1, Z_2\}$ are:
 - $(\hat{X}, \hat{X}), (\hat{X}, QS), (\hat{X}, KS), (\hat{X}, VS),$
 - $(\hat{X}, QH), (\hat{X}, KH), (\hat{X}, VH),$
 - $(VS, \hat{X}), (VS, QS), (VS, KS), (VS, VS),$
 - $(VH, \hat{X}), (VH, QH), (VH, KH), (VH, VH),$
 - $(QS, \hat{X}), (QS, QS), (QS, VS),$
 - $(QH, \hat{X}), (QH, QH), (QH, VH),$
 - $(KS, \hat{X}), (KS, KS), (KS, VS),$
 - $(KH, \hat{X}), (KH, KH), (KH, VH),$
2. **Stage 2 (Refinement for Z_3):** We identify the top-4 performing configurations from Stage 1 and optimize Z_3 across its remaining candidates.

All main experimental results reported in the manuscript are based on the optimal hyperparameter configuration derived from this two-stage optimization. Because the hyperparameter optimization procedure is defined deterministically, the final results are free from random variation.

C Computational Complexity Analysis

Let N denote the number of input tokens, d the feature dimension, H the number of attention heads, and $d_h = d/H$ the feature dimension per head. Let r denote the number of removed tokens and ρ the pivot proportion, yielding

$$N_{\text{pivot}} = \lceil (N - r)\rho \rceil.$$

Algorithm 1 consists of four primary stages:

Table 5: Hyperparameter configurations for fine-tuning downstream EEG foundation models. Pre-trained checkpoints for TUAB and TUEV are provided by TFM-Tokenizer.

Model Dataset	LaBraM					EEGPT				
	TUAB	TUEV	ISRUC	EarEEG	EEGMAT	TUAB	TUEV	ISRUC	EarEEG	EEGMAT
layers			12					8		
initial tokens	230	115	180	120	64	20	20	6	4	16
optimizer			AdamW					AdamW		
scheduler			Cosine			Cosine		OneCycleLR		
seed			0			0		7		
Batch Size			256		377	256		32		
epoch			50			50		40		
warm-up			1			5		8		
lr			5e-4			5e-4		4e-4		
weight decay			0.05			0.05		0.01		
layer decay			0.65			0.65		0		
drop path			0.1			0.1	0.2	0		

Model Dataset	BIOT					TFM-Tokenizer				
	TUAB	TUEV	ISRUC	EarEEG	EEGMAT	TUAB	TUEV	ISRUC	EarEEG	EEGMAT
layers			4					4		
initial tokens	304	144	354	236	112	304	144	354	236	112
optimizer			Adam			/		AdamW		
scheduler			/			/		Custom		
seed			12345			/		5		
Batch Size			512		754	/		256		377
epoch			25			/		50		
warm-up			/			/		5		
lr			5e-4			/		1e-3		

Pivot Selection. The pivot score is calculated by computing the ℓ_2 -norm of each token representation and averaging across attention heads, incurring a complexity of

$$\mathcal{O}(Nd).$$

Selecting the top- N_{pivot} pivot indices introduces negligible computational cost compared to feature transformation.

Redundant Token Identification. Rather than computing pairwise similarities between every pivot and non-pivot token, ZIPBrain aggregates all pivot representations into a single prototype via mean pooling. Non-pivot tokens are subsequently compared against this single prototype, reducing complexity to linear scale:

$$\mathcal{O}(Nd).$$

Target Token Matching. The matching stage computes pairwise similarity scores between the r selected source tokens and the $(N - r)$ retained tokens, resulting in

$$\mathcal{O}(r(N - r)d).$$

Feature Aggregation. The scatter-reduce operation and subsequent feature norm rescaling are both linear in the token count:

$$\mathcal{O}(Nd).$$

Overall Complexity. Combining all stages yields an overall computational complexity of

$$\mathcal{O}(Nd + r(N - r)d),$$

which is bounded by $\mathcal{O}(N^2d)$ in the worst-case scenario.

Because redundancy scoring and feature aggregation operate with linear complexity relative to the sequence length, the quadratic term in ZIPBrain originates solely from the target matching stage.

D Additional Results

In addition to the AUROC, PR-AUC, Cohen’s Kappa, and Weighted-F1 metrics reported in the main paper, we provide Accuracy (Acc) and Balanced Accuracy (BAcc) here for completeness. Table 6 details the classification performance under the maximum available compression ratios across models. Table 7 reports performance across varying compression ratios for TUAB evaluated on BIOT and TUEV evaluated on LaBraM.

E Limitations

While ZIPBrain effectively reduces the sequence length processed by subsequent Transformer layers, the compression module itself introduces minor computational overhead. In particular, the target token matching stage requires pairwise similarity computations between the selected source tokens and the retained tokens, leading to quadratic complexity relative to the participating tokens. Nevertheless, this overhead is substantially lower than that of full self-attention, as matching is restricted to a small subset of tokens after redundancy identification. Consequently, real-world inference latency remains significantly reduced. Exploring linear-

Model	Methods	TUAB		EEGMAT		TUEV		ISRUC		EarEEG	
		Acc	BAcc	Acc	BAcc	Acc	BAcc	Acc	BAcc	Acc	BAcc
LaBraM	Original ¹	0.8221	0.8206	0.5655	0.5655	0.8238	0.6585	0.8003	0.7717	0.5951	0.4463
	EVIT	0.7716	0.7687	0.5793	0.5793	0.8120	0.4357	0.6908	0.6641	0.4226	0.3566
	ToMe	0.8108	0.8036	0.5724	0.5724	0.8113	0.5975	0.6982	0.6684	0.4728	0.3869
	ToFU	0.8150	0.8085	0.5828	0.5828	0.8133	0.6025	0.7183	0.6866	0.4759	0.3855
	DART	0.7918	0.7918	0.5621	0.5621	0.7642	0.5468	0.7445	0.7020	0.5326	0.3934
	Ours	0.8041	<u>0.8053</u>	0.6172	0.6172	0.8269	0.6212	0.7518	0.7244	0.5802	0.4607
EEGPT	Original ¹	0.8012	0.7992	0.6034	0.6034	0.8014	0.5411	0.7383	0.7141	0.6196	0.4675
	EVIT ^{mean}	0.8000	0.7982	0.6138	0.6138	0.8053	0.5316	0.7347	0.7112	0.6009	0.4513
	ToMe	0.7998	0.7980	0.6207	0.6207	0.8008	0.5315	0.7245	0.7028	0.6056	0.4457
	ToFU	0.7995	0.7977	0.6207	0.6207	0.8011	0.5311	0.7269	0.7041	0.6050	0.4464
	DART	<u>0.7999</u>	0.7985	<u>0.6172</u>	<u>0.6172</u>	0.8029	0.5378	0.7424	0.7162	0.6131	0.4530
	Ours	<u>0.7989</u>	0.7974	0.6138	0.6138	<u>0.8018</u>	0.5527	<u>0.7402</u>	<u>0.7143</u>	<u>0.6121</u>	0.4571
BIOT	Original ¹	0.8081	0.8009	0.6276	0.6276	0.7274	0.5059	0.8132	0.7776	0.5802	0.4136
	EVIT ^{mean}	0.7913	0.7900	0.6138	0.6138	0.7007	0.4959	0.6707	0.6463	0.6236	0.5154
	ToMe	0.8073	0.8014	0.6448	0.6448	0.7331	0.4941	<u>0.7978</u>	0.7641	0.5904	0.4210
	ToFU	0.8073	0.8010	0.6552	0.6552	0.7360	0.4881	0.7987	0.7617	0.5863	0.4165
	DART	0.7907	<u>0.7876</u>	0.6414	0.6414	0.7013	0.4419	0.7180	0.6745	0.4759	0.3440
	Ours	0.7989	0.7974	0.6138	0.6138	0.8018	0.5527	0.7402	0.7143	<u>0.6121</u>	<u>0.4571</u>
TFM	Original ¹	0.8208	0.8144	0.5862	0.5862	0.7696	0.5388	0.7798	0.7391	0.5703	0.4247
	EVIT	0.8000	0.7979	0.5931	0.5931	0.7462	0.3643	0.7500	0.7149	0.6019	0.4402
	ToMe	0.8070	<u>0.7994</u>	0.6172	0.6172	0.7659	0.4615	0.7624	0.7256	0.5442	0.4280
	ToFU	0.8080	0.7998	0.6138	0.6138	0.7651	0.4724	0.7655	0.7294	0.5377	0.4228
	DART	0.7935	0.7893	0.5724	0.5724	0.7505	0.5060	0.7625	0.7226	0.4626	0.3486
	Ours	<u>0.8076</u>	0.7977	0.6207	0.6207	0.7659	0.5399	0.7818	0.7376	0.6206	0.4802

Experiment results with r , the number of tokens to be reduced per reduction, set to the biggest integer smaller than $\frac{N}{l}$. N is the number of total tokens and l is set to 4 for BIOT, TFM and EEGPT and 12 for LaBraM. For BIOT and TFM, which consist of 4 layers of encoder, and LaBraM, which consists of 12 layers of encoder, we insert reductions in every layer. For EEGPT, which consists of 8 layer of encoder, reductions are insert at the 2nd, 4th, 6th and 8th layer. All hyper-parameters are set based on recommendations from original works. Compression ratio is defined as $(N - l \cdot r)/N$.

¹ We include experiment results without token compression as Original.

Table 6: Performance comparison under the maximum available compression ratios.

complexity approximations for the matching stage while preserving compression also represents a promising direction for future research.

Dataset	Method	20%		40%		60%		80%	
		Acc	BAcc	Acc	BAcc	Acc	BAcc	Acc	BAcc
TUAB (BIOT)	Original ¹	0.8081	0.8009	0.8081	0.8009	0.8081	0.8009	0.8081	0.8009
	ToMe	0.8090	0.8020	0.8087	0.8018	0.8086	0.8021	0.8082	0.8022
	ToFU	0.8092	0.8022	0.8086	0.8017	0.8080	0.8014	0.8092	0.8029
	EViT	0.8091	0.8028	0.8064	0.8011	0.8039	0.8000	0.7989	0.7966
	DART	0.8095	0.8028	0.8085	0.8025	0.8052	0.8002	0.7979	0.7943
	Ours	0.8093	<u>0.8026</u>	0.8100	0.8036	0.8102	0.8045	0.8106	0.8061
Dataset	Method	20%		40%		60%		80%	
		Acc	BAcc	Acc	BAcc	Acc	BAcc	Acc	BAcc
TUEV (LaBraM)	Original ¹	0.8238	0.6585	0.8238	0.6585	0.8238	0.6585	0.8238	0.6585
	ToMe	0.8185	0.6602	0.8180	0.6608	0.8064	0.6407	0.8071	0.6366
	ToFU	0.8185	0.6603	0.8211	<u>0.6563</u>	0.8100	0.6484	0.8144	0.6319
	EViT	0.8243	0.6480	0.8227	0.6021	0.8206	0.5221	0.8155	0.4542
	DART	0.8187	0.6531	0.8078	0.6355	0.7930	0.6179	0.7820	0.5704
	Ours	0.8356	0.6607	0.8340	0.6541	0.8283	0.6542	0.8237	0.6397

¹ We show results without token compression for easier comparison.

Table 7: Performance evaluation across varying token compression ratios.

Algorithm 1: ZIPBrain Token Compression Pipeline

Input: Token features $\hat{\mathbf{X}} \in \mathbb{R}^{N \times d}$; Metric representations $\mathbf{Z}_1, \mathbf{Z}_2, \mathbf{Z}_3 \in \mathbb{R}^{N \times H \times d_h}$; Token budget r ; Pivot proportion ρ .

Output: Merged token features $\hat{\mathbf{X}}' \in \mathbb{R}^{(N-r) \times d}$.

```
1:  $N_{\text{pivot}} \leftarrow \min(\max(\lceil (N-r) \cdot \rho \rceil, 1), N-r)$ 
2: // Step 1: Pivot Token Selection
3:  $S_0 \leftarrow \text{Mean}_{\text{heads}}(\|\mathbf{Z}_1\|_2)$ 
4:  $I_{\text{pivot}} \leftarrow \text{TopK\_Indices}(S_0, N_{\text{pivot}})$ 
5:  $I_{\text{non\_pivot}} \leftarrow \{1, \dots, N\} \setminus I_{\text{pivot}}$ 
6: // Step 2: Redundancy Scoring & Source Selection
7:  $\hat{\mathbf{Z}}_2 \leftarrow \text{Normalize}(\mathbf{Z}_2, L_2)$ 
8:  $\mathbf{T}_{\text{pivot}} \leftarrow \text{Gather}(\hat{\mathbf{Z}}_2, I_{\text{pivot}}), \quad \mathbf{T}_{\text{non\_pivot}} \leftarrow$   

    $\text{Gather}(\hat{\mathbf{Z}}_2, I_{\text{non\_pivot}})$ 
9:  $S_{\text{dup}} \leftarrow \text{Mean}_{\text{heads}}(\mathbf{T}_{\text{non\_pivot}} \cdot \text{Mean}_{\text{tokens}}(\mathbf{T}_{\text{pivot}}))$ 
10:  $I_{\text{src\_rel}} \leftarrow \text{TopK\_Indices}(S_{\text{dup}}, r)$ 
11:  $I_{\text{src}} \leftarrow \text{Gather}(I_{\text{non\_pivot}}, I_{\text{src\_rel}})$ 
12:  $I_{\text{left}} \leftarrow I_{\text{pivot}} \cup (I_{\text{non\_pivot}} \setminus I_{\text{src}})$ 
13: // Step 3: Target Token Matching
14:  $\hat{\mathbf{Z}}_3 \leftarrow \text{Normalize}(\mathbf{Z}_3, L_2)$ 
15:  $\mathbf{T}_{\text{src}} \leftarrow \text{Gather}(\hat{\mathbf{Z}}_3, I_{\text{src}}), \quad \mathbf{T}_{\text{left}} \leftarrow \text{Gather}(\hat{\mathbf{Z}}_3, I_{\text{left}})$ 
16:  $S_{\text{tgt}} \leftarrow \text{Mean}_{\text{heads}}(\mathbf{T}_{\text{left}} \cdot \mathbf{T}_{\text{src}}^T)$ 
17:  $I_{\text{sim}} \leftarrow \text{argmax}(S_{\text{tgt}}, \text{dim} = \text{left})$ 
18:  $I_{\text{tar}} \leftarrow \text{Gather}(I_{\text{left}}, I_{\text{sim}})$ 
19: // Step 4: Feature Aggregation & Rescaling
20: Define  $\text{Merge}(\mathbf{V}, \text{op})$ :
21:    $\mathbf{V}_{\text{tar}} \leftarrow \text{ScatterReduce}(\mathbf{V}, \text{indices} = I_{\text{tar}}, \text{src} =$   

    $\mathbf{V}[I_{\text{src}}], \text{reduce} = \text{op})$ 
22:   return  $\mathbf{V}_{\text{tar}}[I_{\text{left}}]$ 
23:  $\mathbf{L} \leftarrow \|\hat{\mathbf{X}}\|_2$ 
24:  $\mathbf{L}' \leftarrow \text{Merge}(\mathbf{L}, \text{"amax"})$ 
25:  $\hat{\mathbf{X}}_{\text{merged}} \leftarrow \text{Merge}(\hat{\mathbf{X}}, \text{"sum"})$ 
26:  $\hat{\mathbf{X}}' \leftarrow \frac{\hat{\mathbf{X}}_{\text{merged}}}{\|\hat{\mathbf{X}}_{\text{merged}}\|_2} \odot \mathbf{L}'$ 
27: return  $\hat{\mathbf{X}}'$ 
```
